

---

# DFSS Fine-Tuning Studio

## Theoretischer Hintergrund & Dokumentation

---

Dieses Dokument beschreibt den theoretischen Hintergrund jedes Schrittes im Jupyter-Notebook **DFSS\_FineTuning\_Studio.ipynb**. Es erklart, wie grosse Sprachmodelle funktionieren, warum und wie Fine-Tuning durchgefuehrt wird, welche mathematischen Grundlagen hinter LoRA stehen, und wie das fertige Modell quantisiert und lokal betrieben wird.

| Kapitel | Thema                                | Notebook-Bezug             |
|---------|--------------------------------------|----------------------------|
| 1       | Large Language Models (LLMs)         | Hintergrund                |
| 2       | Transformer-Architektur              | Modell-Verstaendnis        |
| 3       | Tokenisierung                        | Cell 3: format_examples()  |
| 4       | Fine-Tuning: Motivation und Methoden | Cell 3: run_training()     |
| 5       | LoRA: Low-Rank Adaptation            | Cell 3: LoraConfig()       |
| 6       | Trainingstheorie                     | Cell 3: TrainingArguments  |
| 7       | Datensatz-Design                     | Cell 3: load_dataset()     |
| 8       | Quantisierung und GGUF               | Cell 3: convert_to_gguf()  |
| 9       | Ollama: Lokale Inferenz              | Cell 3: create_modelfile() |
| 10      | Evaluation und Qualitaet             | Cell 4: Chat / Quiz        |

---

# 1. Large Language Models (LLMs)

---

## 1.1 Was ist ein Sprachmodell?

Ein Sprachmodell ist ein statistisches Modell, das die Wahrscheinlichkeit einer Sequenz von Woertern berechnet. Gegeben eine Folge von Woertern  $w_1, w_2, \dots, w_{t-1}$ , schätzt das Modell die Wahrscheinlichkeit des naechsten Wortes  $w_t$ :

$$P(w_t | w_1, w_2, \dots, w_{t-1})$$

Moderne LLMs wie DeepSeek-R1 verwenden neuronale Netze mit Milliarden von Parametern, um diese bedingte Wahrscheinlichkeit zu modellieren. Die Textgenerierung erfolgt autoregressiv: Das Modell erzeugt ein Token nach dem anderen, wobei jedes neue Token auf allen bisherigen Tokens basiert.

## 1.2 Groessenordnungen

Die Modellgroesse wird in Parametern gemessen. Jeder Parameter ist ein trainierbares Gewicht im neuronalen Netz:

| Modell             | Parameter         | RAM-Bedarf (float32) | RAM-Bedarf (4-bit) |
|--------------------|-------------------|----------------------|--------------------|
| DeepSeek-R1 1.5B   | 1,5 Milliarden    | ~6 GB                | ~1,5 GB            |
| DeepSeek-R1 7B     | 7 Milliarden      | ~28 GB               | ~4,5 GB            |
| DeepSeek-R1 8B     | 8 Milliarden      | ~32 GB               | ~5 GB              |
| GPT-4 (geschaetzt) | ~1.800 Milliarden | Nicht lokal          | Nicht lokal        |

Im Notebook verwenden wir das **1.5B-Modell**, da es auf einem Laptop-CPU mit 16 GB RAM laeuft. Das 7B-Modell benoetigt 32+ GB RAM.

## 1.3 Vortrainierung vs. Fine-Tuning

LLMs durchlaufen zwei Phasen:

- **Vortrainierung (Pre-Training):** Das Modell wird auf riesigen Textkorpora (Billionen von Tokens) trainiert, um allgemeines Sprachwissen zu erlernen. Dies dauert Wochen auf Hunderten von GPUs und kostet Millionen Euro. DeepSeek-R1 wurde auf einer Mischung aus Webtext, Buechern und Code vortrainiert.
- **Fine-Tuning:** Das vortrainierte Modell wird auf einem kleinen, aufgabenspezifischen Datensatz weiter trainiert. Hier lernt es, auf eine bestimmte Art zu antworten (z.B. als Statistik-Tutor). Dies ist unser Ansatz: Wir passen DeepSeek-R1 mit 193 DFSS-Fragen an.

## 2. Die Transformer-Architektur

### 2.1 Ueberblick

Die Transformer-Architektur (Vaswani et al., 2017) ist die Grundlage aller modernen LLMs. Im Gegensatz zu fruheren Ansaetzen (RNNs, LSTMs) verarbeitet der Transformer alle Tokens einer Sequenz **parallel** mithilfe des **Self-Attention-Mechanismus**.

DeepSeek-R1 1.5B besteht aus folgenden Komponenten:

| Komponente             | Wert    | Erklaerung                           |
|------------------------|---------|--------------------------------------|
| Schichten (Layers)     | 28      | Gestapelte Transformer-Blöcke        |
| Attention-Köpfe        | 16      | Parallele Aufmerksamkeitsmechanismen |
| Embedding-Dimension    | 2048    | Grösse der Token-Darstellung         |
| Feed-Forward-Dimension | 5632    | Breite des MLP-Netzwerks             |
| Kontextfenster         | 131.072 | Maximale Eingabelänge (Tokens)       |
| Vokabular              | 151.936 | Anzahl bekannter Tokens              |

### 2.2 Self-Attention

Der Self-Attention-Mechanismus berechnet fuer jedes Token, wie stark es auf jedes andere Token in der Sequenz 'achten' soll. Fuer jedes Token werden drei Vektoren berechnet:

- **Query (Q):** 'Was suche ich?'
- **Key (K):** 'Was biete ich an?'
- **Value (V):** 'Welche Information trage ich?'

Die Aufmerksamkeitsgewichte werden durch das Skalarprodukt von Q und K berechnet, normalisiert durch die Wurzel der Dimension:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$$

Dabei ist  $d_k$  die Dimension der Key-Vektoren. Die Softmax-Funktion normalisiert die Gewichte auf Summe 1. **Multi-Head Attention** fuehrt diese Berechnung parallel mit mehreren Koepfen durch, wobei jeder Kopf andere Aspekte der Beziehungen erfasst.

### 2.3 Feed-Forward-Netzwerk

Nach der Attention-Schicht durchlauft jedes Token ein positionsweises Feed-Forward-Netzwerk (MLP) mit zwei linearen Transformationen und einer Aktivierungsfunktion:

$$\text{FFN}(x) = W_2 \times \text{activation}(W_1 \times x + b_1) + b_2$$

DeepSeek-R1 verwendet **SwiGLU** als Aktivierungsfunktion, eine Variante die gegenueber dem klassischen ReLU bessere Trainingseigenschaften aufweist. Die Gewichtsmatrizen  $W_1$ ,  $W_2$  (gate\_proj, up\_proj, down\_proj) sind die groessten Parameter im Modell und ein Hauptziel fuer LoRA-Adaption.

## 2.4 Projektionsmatrizen im Detail

Die folgenden Matrizen bilden den Kern des Transformers und sind die **target\_modules** fuer LoRA im Notebook:

| Matrix    | Funktion                         | Groesse (1.5B) | In LoRA? |
|-----------|----------------------------------|----------------|----------|
| q_proj    | Query-Projektion in Attention    | 2048 x 2048    | Ja       |
| k_proj    | Key-Projektion in Attention      | 2048 x 256     | Ja       |
| v_proj    | Value-Projektion in Attention    | 2048 x 256     | Ja       |
| o_proj    | Output-Projektion nach Attention | 2048 x 2048    | Ja       |
| gate_proj | Gate im SwiGLU-FFN               | 2048 x 5632    | Ja       |
| up_proj   | Up-Projektion im FFN             | 2048 x 5632    | Ja       |
| down_proj | Down-Projektion im FFN           | 5632 x 2048    | Ja       |

Tabelle: Projektionsmatrizen und ihre Rolle. Alle 7 werden per LoRA adaptiert.

---

## 3. Tokenisierung

---

### 3.1 Was ist Tokenisierung?

LLMs arbeiten nicht mit ganzen Woertern, sondern mit **Tokens** — Teilwoertern, die durch einen Tokenizer in ganzzahlige IDs umgewandelt werden. Der Tokenizer von DeepSeek-R1 verwendet **Byte-Pair Encoding (BPE)** mit einem Vokabular von 151.936 Tokens.

Beispiel: 'Standardabweichung' wird moeglicherweise in ['Standard', 'ab', 'weich', 'ung'] aufgeteilt, wobei jedes Teilteil eine eindeutige ID erhaelt.

### 3.2 Chat-Template

DeepSeek-R1 verwendet ein spezielles Chat-Format mit Rollen-Markierungen:

```
<begin_of_sentence>SYSTEM_PROMPT
<|User|>Was ist Varianz?
<|Assistant|>Die Varianz ist...<end_of_sentence>
```

Im Notebook wird dies durch `tokenizer.apply_chat_template()` automatisch formatiert. Die korrekte Formatierung ist entscheidend: Wenn das Trainingsformat nicht zum Inferenzformat passt, produziert das Modell unsinnige Ausgaben.

### 3.3 Sequenzlaenge und Abschneidung

Der Parameter **MAX\_SEQ\_LENGTH = 512** begrenzt die maximale Laenge einer Trainingsinstanz auf 512 Tokens. Laengere Beispiele werden abgeschnitten (truncation). Dies betrifft ~5% der Q&A-Paare. Ein groesserer Wert (z.B. 1024) erfasst mehr Kontext, benoetigt aber proportional mehr Speicher und Rechenzeit.

## 4. Fine-Tuning: Motivation und Methoden

### 4.1 Warum Fine-Tuning?

Ein vortrainiertes LLM hat breites Allgemeinwissen, aber keine Spezialisierung. Fine-Tuning passt das Modell an eine spezifische Aufgabe oder Domaene an:

- **Domaenen-Anpassung:** Das Modell lernt DFSS-Terminologie, Formeln und Denkweisen.
- **Stil-Anpassung:** Antworten im gewuenschten Format (Schritt-fuer-Schritt, Unicode-Formeln).
- **Fakten-Verankerung:** Das Modell priorisiert Wissen aus dem Trainingsmaterial.

### 4.2 Methoden des Fine-Tuning

| Methode             | Parameter trainiert  | RAM-Bedarf            | Qualitaet |
|---------------------|----------------------|-----------------------|-----------|
| Full Fine-Tuning    | Alle (100%)          | Sehr hoch (4x Modell) | Beste     |
| LoRA (unser Ansatz) | 0,5-2%               | Moderat (1,2x Modell) | Sehr gut  |
| QLoRA               | 0,5-2% (4-bit Basis) | Gering (0,5x Modell)  | Gut       |
| Prefix Tuning       | <0,1%                | Gering                | Mittel    |
| Prompt Tuning       | <0,01%               | Minimal               | Begrenzt  |

Wir verwenden **LoRA** — den besten Kompromiss zwischen Qualitaet und Ressourcenbedarf fuer CPU-Training. Nur ~0,5% der Parameter werden trainiert, waehrend die Modellqualitaet fast dem Full Fine-Tuning entspricht.

### 4.3 Verlustfunktion (Loss)

Beim Training eines Sprachmodells wird die **Cross-Entropy-Verlustfunktion** minimiert. Fuer eine Sequenz von T Tokens berechnet sich der Verlust als:

$$L = -(1/T) \times \sum_{t=1}^T \log P(w_t \mid w_1, \dots, w_{t-1})$$

Der Verlust misst, wie gut das Modell das naechste Token vorhersagt. Ein kleinerer Verlust bedeutet bessere Vorhersagen. Typische Verlaeufe: Startverlust ~3-4, Endverlust ~1-2 nach Fine-Tuning.

---

## 5. LoRA: Low-Rank Adaptation

---

### 5.1 Die Kernidee

LoRA (Hu et al., 2021) basiert auf der Beobachtung, dass die Gewichtsänderungen beim Fine-Tuning einen **niedrigen Rang** haben. Statt eine grosse Gewichtsmatrix  $W$  direkt zu ändern, zerlegen wir die Änderung in zwei kleine Matrizen:

$$W' = W + \Delta W = W + B \times A$$

Dabei gilt:

- **W**: Ursprüngliche Gewichtsmatrix (eingefroren, nicht trainiert). Dimension:  $d \times d$
- **A**: Niedrigrang-Matrix, Dimension:  $d \times r$  (zufällig initialisiert)
- **B**: Niedrigrang-Matrix, Dimension:  $r \times d$  (mit Nullen initialisiert)
- **r**: Rang (im Notebook:  $r = 16$ ). Steuert die Kapazität der Adaption.

### 5.2 Parameterersparnis

Für eine Gewichtsmatrix der Grösse  $d \times d$  mit  $d = 2048$ :

- Original:  $d \times d = 2048 \times 2048 = \mathbf{4.194.304}$  Parameter
- LoRA ( $r=16$ ):  $d \times r + r \times d = 2048 \times 16 + 16 \times 2048 = \mathbf{65.536}$  Parameter
- Ersparnis:  $65.536 / 4.194.304 = \mathbf{1,56\%}$  der ursprünglichen Parameter

Über alle 7 Zielmatrizen und 28 Schichten ergibt sich die Gesamtzahl trainierbarer Parameter, die im Notebook als `model.print_trainable_parameters()` angezeigt wird.

### 5.3 Alpha und Skalierung

Der Parameter **lora\_alpha** (im Notebook: 32) skaliert die LoRA-Änderung:

$$W' = W + (\alpha/r) \times B \times A$$

Das Verhältnis  $\alpha/r$  steuert, wie stark die LoRA-Adaption die Originalgewichte beeinflusst. Ein höheres Verhältnis (z.B.  $\alpha=32, r=16 \rightarrow$  Faktor 2) bedeutet stärkere Anpassung. Typische Wahl:  $\alpha = 2 \times r$ .

### 5.4 Dropout

LoRA-Dropout (im Notebook: 0,05) deaktiviert zufällig 5% der LoRA-Verbindungen während des Trainings. Dies verhindert Überanpassung (Overfitting) an den kleinen Trainingsdatensatz und verbessert die Generalisierungsfähigkeit des Modells.

### 5.5 Zusammenfassung der LoRA-Konfiguration im Notebook

```
LoraConfig(
```

---

```
r=16,                # Rang der Adaption
lora_alpha=32,        # Skalierungsfaktor
lora_dropout=0.05,    # Regularisierung
target_modules=[      # Welche Matrizen adaptiert werden
    "q_proj", "k_proj", "v_proj", "o_proj",
    "gate_proj", "up_proj", "down_proj",
],
bias="none",          # Bias-Terme nicht trainieren
task_type=CAUSAL_LM,  # Autoregressives Sprachmodell
)
```



## 6. Trainingstheorie

### 6.1 Gradientenabstieg

Das Training optimiert die LoRA-Parameter durch iterativen Gradientenabstieg. In jedem Schritt:

- 1. Ein Batch von Trainingsbeispielen wird dem Modell praesentiert.
- 2. Der Verlust (Cross-Entropy) wird berechnet.
- 3. Die Gradienten (partielle Ableitungen des Verlusts nach den Parametern) werden berechnet.
- 4. Die Parameter werden in Richtung des negativen Gradienten aktualisiert.

$$\theta_{t+1} = \theta_t - \eta \times \nabla L(\theta_t)$$

Dabei ist  $\eta$  die Lernrate (learning rate) und  $\nabla L$  der Gradient des Verlusts.

### 6.2 AdamW-Optimierer

Das Notebook verwendet den **AdamW-Optimierer** (Loshchilov & Hutter, 2017), der drei Vorteile gegenueber einfachem Gradientenabstieg bietet:

- **Adaptive Lernrate:** Jeder Parameter erhaelt seine eigene effektive Lernrate, basierend auf der Historie seiner Gradienten.
- **Momentum:** Gradientenrichtungen werden ueber mehrere Schritte gemittelt, was die Konvergenz beschleunigt und Oszillationen reduziert.
- **Weight Decay:** L2-Regularisierung verhindert, dass Parameter zu grosse Werte annehmen (Ueberanpassung).

### 6.3 Trainingsparameter im Detail

| Parameter              | Wert | Bedeutung                                             |
|------------------------|------|-------------------------------------------------------|
| epochs                 | 3    | Anzahl der Durchlaeuft ueber den gesamten Datensatz   |
| batch_size             | 1    | Beispiele pro Vorwaertsdurchlauf (begrenzt durch RAM) |
| gradient_accumulation  | 8    | Effektive Batchgrosse = 1 x 8 = 8                     |
| learning_rate          | 2e-4 | Schrittweite der Parameteraktualisierung              |
| warmup_steps           | 10   | Langsamer Anstieg der LR am Anfang                    |
| weight_decay           | 0.01 | Staerke der L2-Regularisierung                        |
| gradient_checkpointing | True | Spart ~30% RAM auf Kosten der Geschwindigkeit         |

### 6.4 Gradient Accumulation

---

Da der Batch-Size auf CPU auf 1 begrenzt ist, simulieren wir einen groesseren Batch durch **Gradient Accumulation**. Die Gradienten von 8 einzelnen Beispielen werden aufaddiert, bevor ein Optimierungsschritt durchgefuehrt wird. Der effektive Batch ist also  $1 \times 8 = 8$ , was stabileres Training ergibt als Batch-Size 1 allein.

## 6.5 Gradient Checkpointing

Normalerweise speichert das Training alle Zwischenergebnisse (Aktivierungen) fuer die Rueckwaertspropagation. Gradient Checkpointing verwirft die meisten Zwischenergebnisse und berechnet sie bei Bedarf neu. Dies **reduziert den RAM-Bedarf um ~30%**, verlangsamt aber das Training um ~20%. Auf CPUs mit begrenztem RAM ist dieser Kompromiss essentiell.

---

## 7. Datensatz-Design und Qualitaet

---

### 7.1 Herkunft des Datensatzes

Der Trainingsdatensatz wurde von Claude (Anthropic) generiert, basierend auf dem vollstaendigen 369-seitigen DFSS-Statistik-Skript. Die Generierung folgte einem mehrstufigen Prozess:

- **Extraktion:** Volltext des PDFs wurde seitenweise mit pdfplumber extrahiert.
- **Chunking:** Der Text wurde in 253 semantische Abschnitte aufgeteilt.
- **Q&A-Generierung:** Claude erzeugte pro Abschnitt 5-6 diverse Frage-Antwort-Paare.
- **Qualitaetskontrolle:** Mehrfache Validierung: Erzeugung → Kritik → Filterung.
- **Deduplizierung:** Aehnliche Fragen wurden durch Hash-Vergleich entfernt.

### 7.2 Diversitaet und Balance

Ein guter Fine-Tuning-Datensatz ist **divers und ausgewogen**. Unser Datensatz umfasst:

| Dimension             | Verteilung                                                                                                                            | Zweck                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| 8 Fragetypen          | Definition 26%, Konzeptuell 22%, Berechnungen 18%, Anwendungsfragen 14%, Vergleich 10%, Zusammenfassung 6%, Bewertung 4%, Sonstige 2% | Diversitaet der Denkweisen trainieren |
| 3 Schwierigkeitsgrade | Leicht 21%, Mittel 58%, Schwer 21%                                                                                                    | Breites Antwortspektrum               |
| 13 Themenbereiche     | Wahrscheinlichkeit, Verteilungen, Tests, Hypothesen, etc.                                                                             | Vollstaendige Abdeckung               |
| Antwortlaengen        | 80-500 Zeichen, Durchschnitt 280                                                                                                      | Praezise aber vollstaendige Antworten |

### 7.3 ChatML-Format

Jedes Trainingsbeispiel ist ein Chat mit drei Nachrichten:

```
{"messages": [  
  {"role": "system",    "content": "Du bist ein Statistik-Tutor..."},  
  {"role": "user",      "content": "Was ist die Stichprobenvarianz?"},  
  {"role": "assistant", "content": "Die Stichprobenvarianz ist..."}  
]}
```

Das System-Prompt definiert die Rolle des Modells und wird bei jeder Frage mitgeliefert. Dies ist der gleiche Prompt, der spaeter bei der Inferenz verwendet wird.

### 7.4 Warum ist Datensatz-Qualitaet wichtiger als Quantitaet?

Fuer Fine-Tuning kleiner Modelle gilt: **Qualitaet > Quantitaet**. 193 praezise, von Claude validierte Paare sind effektiver als Tausende automatisch generierte Paare mit Fehlern. Jedes fehlerhafte Beispiel 'vergiftet' das Training und kann Halluzinationen verstaerken. Daher wurde jedes Paar auf faktische Korrektheit, Vollstaendigkeit und Klarheit geprueft.

---

## 8. Quantisierung und GGUF-Format

---

### 8.1 Was ist Quantisierung?

Quantisierung reduziert die Präzision der Modellgewichte, um Speicher und Rechenzeit zu sparen. Statt 32-Bit-Gleitkommazahlen (float32) werden niedrigere Präzisionen verwendet:

| Format              | Bits/Gewicht   | Modellgrösse (1.5B) | Qualitätsverlust  |
|---------------------|----------------|---------------------|-------------------|
| float32             | 32             | ~6 GB               | Keiner (Original) |
| float16             | 16             | ~3 GB               | Minimal           |
| int8 (Q8_0)         | 8              | ~1,5 GB             | Sehr gering       |
| Q4_K_M (unser Wahl) | 4,5 (gemischt) | ~1 GB               | Gering            |
| Q4_0                | 4              | ~0,8 GB             | Moderat           |
| Q2_K                | 2,5 (gemischt) | ~0,5 GB             | Spürbar           |

### 8.2 Q4\_K\_M: Unsere Wahl

**Q4\_K\_M** (4-bit, K-quant, Medium) ist ein gemischtes Quantisierungsschema aus llama.cpp. 'K-quant' bedeutet, dass verschiedene Schichten unterschiedliche Präzision erhalten: wichtigere Schichten (Attention) behalten höhere Präzision, während weniger kritische Schichten (FFN) stärker quantisiert werden. 'M' steht für Medium-Balance zwischen Grösse und Qualität.

### 8.3 GGUF-Dateiformat

GGUF (GPT-Generated Unified Format) ist das Standardformat für quantisierte Modelle in llama.cpp und Ollama. Es speichert:

- **Metadaten:** Modellarchitektur, Tokenizer-Konfiguration, Chat-Template
- **Tensor-Daten:** Quantisierte Gewichtsmatrizen aller Schichten
- **Vokabular:** Eingebettetes BPE-Vokabular und Merge-Regeln

Die Konvertierung von HuggingFace-Format zu GGUF erfolgt durch das Skript `convert_hf_to_gguf.py` aus llama.cpp.

### 8.4 Konvertierungspipeline im Notebook

Die Export-Pipeline im Notebook (Tab 'Export') durchläuft drei Schritte:

- **Merge LoRA:** Die LoRA-Adapter ( $B \times A$ ) werden zurück in die Basisgewichte gerechnet:  $W' = W + (\alpha/r) \times B \times A$ . Das Ergebnis ist ein vollständiges Modell ohne Adapter.
- **Convert to GGUF:** Das gemergte HuggingFace-Modell wird in das quantisierte GGUF-Format konvertiert. Die Ausgabe ist eine einzelne Datei (~1 GB für 1.5B Q4\_K\_M).

- 
- **Import to Ollama:** Die GGUF-Datei wird ueber ein Modelfile in Ollama registriert. Danach ist das Modell unter dem Namen 'deepseek-dfss' verfuegbar.

---

## 9. Ollama: Lokale Inferenz

---

### 9.1 Was ist Ollama?

Ollama ist ein Open-Source-Tool fuer lokale LLM-Inferenz. Es verwaltet Modelle, optimiert die CPU/GPU-Nutzung und stellt eine REST-API bereit. Im Gegensatz zu Cloud-APIs (OpenAI, Anthropic) laufen alle Berechnungen lokal — keine Daten verlassen den Computer.

### 9.2 Das Modelfile

Ein Modelfile ist eine Konfigurationsdatei, die Ollama mitteilt, wie das Modell geladen und konfiguriert werden soll:

```
FROM deepseek-dfss-Q4_K_M.gguf

PARAMETER temperature 0.2
PARAMETER num_predict 1024

SYSTEM "Du bist ein Statistik-Tutor fuer DFSS..."
```

- **FROM:** Pfad zur GGUF-Datei (das fine-tuned Modell)
- **PARAMETER temperature:** Steuert die Zufaelligkeit (0.2 = fokussiert und faktisch)
- **PARAMETER num\_predict:** Maximale Anzahl generierter Tokens
- **SYSTEM:** System-Prompt, der die Rolle des Modells definiert

### 9.3 Temperatursteuerung bei der Inferenz

Die Temperatur  $T$  skaliert die Logits vor der Softmax-Funktion:

$$P(\text{Token}_i) = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

- **$T = 0.0$ :** Greedy — immer das wahrscheinlichste Token. Deterministisch, aber repetitiv.
- **$T = 0.2$ :** Unsere Wahl — fokussiert, geringe Zufaelligkeit, ideal fuer Fakten.
- **$T = 1.0$ :** Standard-Sampling — volle Vielfalt, hoehere Kreativitaet.

### 9.4 REST-API

Das Notebook kommuniziert mit Ollama ueber die lokale REST-API (Port 11434). Die Chat-Funktion im Gradio-Tab sendet POST-Anfragen an `http://localhost:11434/api/generate` und streamt die Antwort Token fuer Token zurueck.

---

## 10. Evaluation und Qualitätsbewertung

---

### 10.1 Trainingsverlust

Der primäre Indikator während des Trainings ist der **Trainingsverlust** (Loss). Dieser sollte über die Epochen monoton sinken. Typische Werte:

- Startverlust: ~3.0-4.0 (Modell hat noch nichts gelernt)
- Nach Epoche 1: ~1.5-2.5
- Nach Epoche 3: ~0.8-1.5 (gute Konvergenz)

Wenn der Verlust nicht sinkt: Lernrate ist zu niedrig. Wenn er zu schnell auf ~0 sinkt: Überanpassung (Overfitting) — das Modell memorisiert den Datensatz statt zu generalisieren.

### 10.2 Validierungsverlust

Das Notebook spart 10% des Datensatzes (20 Paare) als Validierungsset aus. Der Validierungsverlust misst die Leistung auf ungesehenen Daten. Wenn der Trainingsverlust sinkt aber der Validierungsverlust steigt, liegt Überanpassung vor. In diesem Fall: weniger Epochen, mehr Dropout, oder mehr Trainingsdaten.

### 10.3 Qualitative Evaluation

Neben dem Verlust ist eine **qualitative Bewertung** essentiell:

- **Chat-Tab:** Fragen stellen und Antworten lesen — sind sie korrekt und hilfreich?
- **Quiz-Tab:** Zufällige Fragen aus dem Trainingsset — stimmen die Modell-Antworten mit den Referenz-Antworten überein?
- **Vergleich:** Basis-Modell vs. Fine-Tuned — zeigt der Chat-Tab (Modellauswahl) den Unterschied?

### 10.4 Grenzen des Ansatzes

Wichtige Einschränkungen, die man kennen sollte:

- **Modellgröße:** Ein 1.5B-Modell hat begrenzte Kapazität. Für komplexe Ableitungen und mehrstufige Berechnungen ist das 7B-Modell überlegen.
- **Datensatzgröße:** 193 Paare sind ausreichend für Stil- und Formatanpassung, aber zu wenig für tiefes Faktenwissen. Das Modell wird weiterhin halluzinieren, wenn es mit Fragen außerhalb des Datensatzes konfrontiert wird.
- **Sprache:** Der Datensatz ist auf Englisch. Für deutsche Antworten müsste ein deutscher Datensatz erstellt und trainiert werden.
- **Keine Bilder:** Das Modell kann keine Plots oder Diagramme interpretieren. Für visuelle Inhalte ist ein RAG-System mit Vision-Modell besser geeignet.

---

*Ende der Dokumentation — DFSS Fine-Tuning Studio*